@Aman_Views

Go Programming Language Handbook

1. Go Fundamentals & Program Structure

→ What is Go ?

Go (also called Golang) is an open-source programming language developed by Google. It is simple, fast, reliable and efficient. Go is designed for modern multi-core computers and large scale systems.

→ Why Go was Created ?

- To improve compilation speed.
- To provide high performance.
- To handle concurrency easily.
- To reduce complex and heavy syntax of other languages.
- To build scalable and maintainable software.

→ Key Features of Go

- Simple and clean syntax
- Compiled language (fast execution)
- Strongly typed and statically compiled
- Built-in concurrency (Goroutines & Channels)
- Lightweight threads (Goroutines)
- Automatic memory management (Garbage Collection)
- Cross-platform support
- Rich standard library
- Good tooling (go build, go test, go fmt, etc.)
- Designed for scalability and performance

→ Go vs Other Languages (High Level)

Feature	Go	Java	Python	C++
Speed	Fast (Compiled)	Fast (JVM)	Slow (Interpreted)	Very Fast
Concurrency	Built-in (Goroutines)	Threads	Limited (GIL)	Threads
Syntax	Simple	Complex	Very Simple	Complex
Memory Mgmt.	Automatic (GC)	Automatic (GC)	Automatic (GC)	Manual + Smart Ptr
Use Case	Cloud, Backend, DevOps, Microservices	Enterprise, Android	AI/ML, Scripting, Web	Systems, Games, High Performance

→ Installing Go

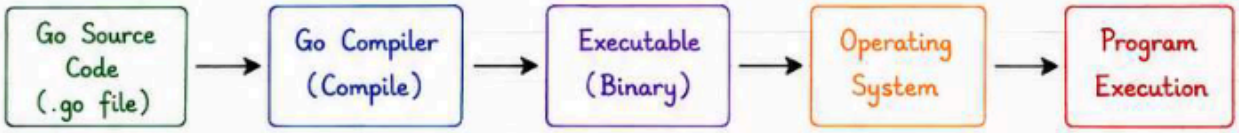
- Download from official site : https://golang.org
- Install for your OS (Windows / macOS / Linux)
- Verify installation

```
Terminal : go version
```

→ First Go Program

```
package main
import "fmt"
func main() {
    fmt.Println("Hello, Go !")
}
```

→ Go Program Structure & Flow



★ Go is compiled, not interpreted.

★ Compilation produces a machine code executable for the target OS.

2. Data Types, Operators & Control Flow

→ Basic Data Types in Go

Category	Types
Integers	int, int8, int16, int32, int64, uint, uint8, uint16, uint32, uint64
Floats	float32, float64
Complex	complex64, complex128
Boolean	bool (true / false)
String	string
Others	byte (alias for uint8) rune (alias for int32, represents Unicode)

→ Variable Declaration

1. Using var keyword

```
var name string = "Aman"
var age int = 21
var price float64 = 99.99
```

2. Short variable declaration (inside function)

```
name := "Aman"
age := 21
isStudent := true
```

Rules for :=

- At least one new variable must be declared.
- Cannot be used outside a function.

→ Constants

```
const PI = 3.14159
const AppName = "GoLang Handbook"
const (
    MaxUser = 100
    MinUser = 1
)
```

→ Zero Values (Default Values)

- int → 0
- float64 → 0.0
- bool → false
- string → "" (empty string)
- pointer, slice, map, func, chan → nil

→ Type Inference

- Go can automatically infer the type of a variable.
- Makes code simpler and cleaner.

```
var count = 10        // int
pi := 3.14            // float64
name := "Go"          // string
```

→ Operators in Go

Arithmetic : + - * / %

Relational : == != < > <= >=

Logical : && || !

Bitwise : & | ^ &^ << >>

Assignment : = += -= *= /= %= etc.

→ Control Flow Statements

1. if - else

```
if age >= 18 {
    fmt.Println("Adult")
} else {
    fmt.Println("Minor")
}
```

2. if - else if - else

```
if marks >= 90 {
    fmt.Println("A Grade")
} else if marks >= 75 {
    fmt.Println("B Grade")
} else {
    fmt.Println("C Grade")
}
```

3. switch

```
switch day {
case 1:
    fmt.Println("Mon")
case 2:
    fmt.Println("Tue")
default:
    fmt.Println("Other")
}
```

4. for loop (Only loop in Go)

```
for i := 1; i <= 5; i++ {
    fmt.Println(i)
}
```

Infinite loop

```
for {
    // do something
}
```

5. range

- With slice / array

```
nums := []int{10, 20, 30}
for index, value := range nums {
    fmt.Println(index, value)
}
```

- With map

```
m := map[string]int{"Aman":21, "Dev":20}
for key, val := range m {
    fmt.Println(key, val)
}
```

6. break & continue

```
for i := 1; i <= 5; i++ {
    if i == 3 {
        continue  // skip 3
    }
    if i == 5 {
        break     // stop at 5
    }
    fmt.Println(i)
}
```

→ Multiple Return Values

```
func add(a int, b int) (int, int) {
    sum := a + b
    return sum, a - b
}
```

→

```
sum, diff := add(10, 4)
fmt.Println(sum, diff) // 14 6
```

Important Notes

- Go has only one looping construct → for.
- No (), {}, or [] like other languages for control flow (except switch).

3. Functions, Pointers & Defer

→ Functions in Go

- A function is a block of code that performs a specific task.
- Go supports multiple return values.
- Functions are first-class citizens.

```go
func add(a int, b int) int {
    return a + b
}
func main() {
    sum := add(10, 20)
    fmt.Println("Sum:", sum)
}
```

Functions can be defined before or after main().

→ Multiple Return Values

```go
func divide(a, b int) (int, int) {
    q := a / b
    r := a % b
    return q, r
}
func main() {
    q, r := divide(17, 5)
    fmt.Println(q, r)  // Output: 3 2
}
```

// quotient
// remainder

→ Named Return Values

```go
func swap(a, b int) (x, y int) {
    x = b
    y = a
    return    // x and y are returned
}
```

→ Variadic Functions

- Accepts zero or more arguments of the same type.

```go
func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}
```

// Usage

```go
fmt.Println(sum(1,2,3)) // 6
fmt.Println(sum(10,20)) // 30
fmt.Println(sum())      // 0
```

→ Recursion

```go
func fact(n int) int {
    if n <= 1 {
        return 1
    }
    return n * fact(n-1)
}
```

// Usage

```go
fmt.Println(fact(5))  // 120
```

→ Pointers in Go

- Go is pass-by-value, but pointers allow functions to modify the original value.
- Use & to get address, * to access value.

Pointer Basics

```go
var x int = 10
p := &x          // & gives address
fmt.Println(p)   // address of x
fmt.Println(*p)  // value at address
*p = 20          // change value
fmt.Println(x)   // 20
```

Pointer in Function

```go
func increment(n *int) {
    *n = *n + 1
}
func main() {
    a := 10
    increment(&a)
    fmt.Println(a)  // 11
}
```

→ Anonymous Functions & Closures

- Anonymous function: function without a name.

```go
fn := func(a, int) int {
    return x * x
}
fmt.Println(fn(5))      // 25
```

- Closures: function that remembers variables from its surrounding scope.

// Closure Example

```go
func adder() func(int) int {
    sum := 0
    return func(x int) int {
        sum += x
        return sum
    }
}
```

// Usage

```go
add := adder()
fmt.Println(add(5))   // 5
fmt.Println(add(10))  // 15
fmt.Println(add(20))  // 35
```

→ Defer, Panic & Recover

- defer: delays the execution of a function until the surrounding function returns.
- panic: stops normal execution.
- recover: regains control of a panicking goroutine.

Defer

```go
func main() {
    fmt.Println("Start")
    defer fmt.Println("Defer 1")
    defer fmt.Println("Defer 2")
    fmt.Println("End")
}
```

// Output
Start
End
Defer 2
Defer 1

Panic

```go
func main() {
    fmt.Println("Before")
    panic("Something went wrong!")
    fmt.Println("After")
}
```

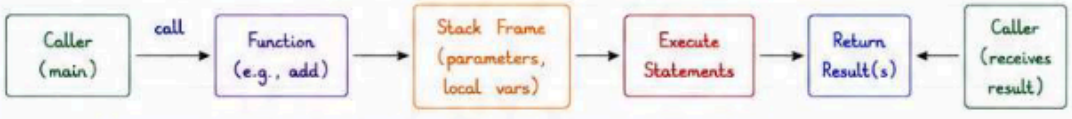
// Output
Before
panic: Something went wrong!
(Program stops here)

Recover

```go
func safe() {
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("Recovered:", r)
        }
    }()
    panic("Oops!")
}
func main() {
    safe()
    fmt.Println("Program continues")
}
```

// Output
Recovered: Oops!
Program continues

How Function Call Works (Conceptual)



★ Each function call creates a stack frame.
★ Parameters and local variables live in that frame.
★ After return, the frame is removed from the stack.

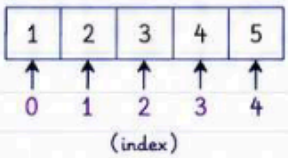
4. Arrays, Slices & Maps

→ Arrays

- Fixed size collection of elements of the same type.
- Size is part of the type.
- Elements are stored in contiguous memory.
- Zero value of array is all elements are zero.

```
var arr [5]int
arr = [5]int{1, 2, 3, 4, 5}
fmt.Println(arr)        // [1 2 3 4 5]
fmt.Println(len(arr)) // 5
```

Array of 5 ints

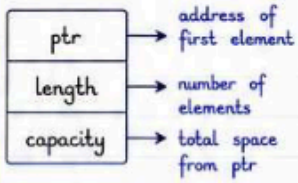


→ Slices

- Slice is a dynamic view of an array.
- Size is not part of the type.
- More powerful, flexible and widely used.
- Internally a slice has 3 parts:
  pointer, length, capacity

```
s := []int{10, 20, 30}
fmt.Println(s)          // [10 20 30]
s = append(s, 40)
fmt.Println(s)          // [10 20 30 40]
fmt.Println(len(s)) // 4
fmt.Println(cap(s)) // 6 (underlying array)
```

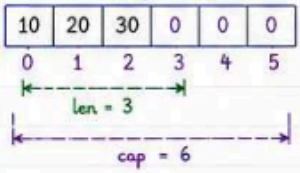
Slice Internals



Example

```
s := []int{10,20,30}
```

Underlying Array



→ Slice Operations

1. Slicing

```
s2 := s[1:3]    // elements from index 1 to 2
fmt.Println(s2)  // [20 30]
```

2. Full Slice Expression

```
s3 := s[1:4:5]  // low : high : max
fmt.Println(len(s3), cap(s3)) // 3 4
```

3. append()

```
s = append(s, 50)
s = append(s, 60, 70)
```

4. make()

```
s4 := make([]int, 2, 5)  // len = 2, cap = 5
```

5. copy()

```
s5 := make([]int, len(s))
copy(s5, s)
```

Important Notes

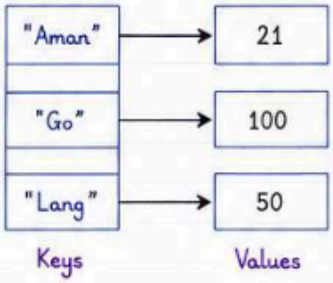
- ★ A slice shares the underlying array.
- ★ Changing one slice may affect another.
- ★ Use append carefully (can create new array).
- ★ nil slice: var s []int (zero value)
- ★ len(s) = number of elements
- ★ cap(s) = capacity from current ptr

→ Maps

- Map is an unordered collection of key-value pairs.
- Key must be of comparable type (==, != allowed).
- Values can be of any type.
- Best for fast lookups.

```
m := make(map[string]int)
m["Aman"] = 21
m["Go"] = 100
fmt.Println(m["Aman"])   // 21
val, ok := m["Go"]
fmt.Println(val, ok)   // 100 true
delete(m, "Go")
fmt.Println(m)   // map[Aman:21]
_, ok = m["Unknown"] // key not found
fmt.Println(ok)   // false
```

Map Visualization



Map Operations

1. Create
   ```
   m := make(map[string]int)
   ```
2. Insert / Update
   ```
   m["Key"] = value
   ```
3. Read
   ```
   val := m["Key"]
   ```
4. Delete
   ```
   delete(m, "Key")
   ```
5. Check if key exists
   ```
   val, ok := m["Key"]
   ```
6. Iterate over map
   ```
   for k, v := range m {
       fmt.Println(k, v)
   }
   ```

→ Key Differences

Feature	Array	Slice	Map
Size	Fixed	Dynamic	Dynamic (key-value)
Type	Size is part of type	Size is not part	Reference type
Access	By index	By index	By key
Zero Value	All zero	nil	nil
Use Case	Fixed size data	Dynamic list	Fast lookups

→ Quick Example Program

```
package main
import "fmt"
func main() {
    // Array
    arr := [3]int{1, 2, 3}
    fmt.Println(arr)
    // Slice
    s := []int{10, 20}
    s = append(s, 30)
    fmt.Println(s)
    // Map
    m := map[string]int{"Aman":21, "Go":100}
    fmt.Println(m["Aman"])
}
```

5. Structs, Methods & Interfaces

→ Structs

- Struct is a user-defined data type that groups related fields of different types.
- It is like a blueprint for creating objects.
- Fields are accessed using dot (.) operator.

```
type Person struct {
    Name    string
    Age     int
    Email   string
}
```

Usage:

```
p := Person{"Aman", 21, "aman@email.com"}
fmt.Println(p.Name)    // Aman
fmt.Println(p.Age)     // 21
fmt.Println(p.Email)   // aman@email.com
```

- Nested Struct Example

```
type Address struct {
    City string
    Pin  int
}
type Employee struct {
    ID      int
    Name    string
    Addr    Address
}
```

Usage:

```
e := Employee{
    ID: 101,
    Name: "Aman",
    Addr: Address{City: "Noida", Pin: 201301},
}
fmt.Println(e.Addr.City)   // Noida
```

→ Methods

- Methods are functions with a receiver.
- They are associated with a type (struct).
- Provide behavior to the data.

Value Receiver

```
func (p Person) Greet() {
    fmt.Println("Hello, I am", p.Name)
}
```

p.Greet()

Pointer Receiver

```
func (p *Person) SetEmail(email string) {
    p.Email = email
}
```

p.SetEmail("new@email.com")

Value vs Pointer Receiver

- Value receiver works on a copy.
- Pointer receiver works on the original data.
- Use pointer receiver when you want to modify the value or avoid copying large structs.

→ Method Sets

Receiver Type	Methods in Method Set
T (value receiver)	Methods with receiver (T) Methods with receiver (*T)
*T (pointer receiver)	Methods with receiver (T) Methods with receiver (*T)

→ Interfaces

- Interface is a contract (set of methods).
- Any type that implements all methods of an interface implicitly satisfies it.
- No explicit implementation required.

```
type Speaker interface {
    Speak()
}
```

There is no "implements" keyword in Go.
If a type has the methods, it implements the interface.

→ Interface Implementation

```
type Dog struct {
    Name    string
}
func (d Dog) Speak() {
}   fmt.Println("Woof! I am", d.Name)
```

Usage:

```
var s Speaker
d := Dog{"Buddy"}
s = d       // Dog implements Speaker
s.Speak()   // Woof! I am Buddy
```

→ Empty Interface (any)

- An interface with zero methods.
- Can hold values of any type.

```
var x any
x = 10
x = "hello"
x = true
```

→ Type Assertion

```
val, ok := x.(string)
if ok {
    fmt.Println("String:", val)
}
```

→ Type Switch

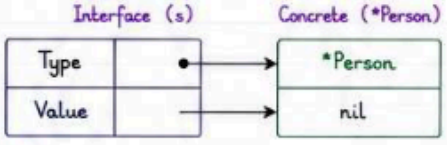
```
func checkType(i any) {
    switch v := i.(type) {
    case int:
        fmt.Println("int:", v)
    case string:
        fmt.Println("string:", v)
    case bool:
        fmt.Println("bool:", v)
    default:
        fmt.Println("unknown type")
    }
}
```

Usage:

```
checkType(10)
checkType("Go")
checkType(true)
checkType(3.14)
```

→ Interface Nil Pitfall

```
var p *Person = nil
var s Speaker = p
```



Interface is NOT nil because its Type is *Person (not nil), even though the Value is nil.

Check:

- s == nil // false
- s == (*Person)(nil) // false
- p == nil // true

Composition over Inheritance

- Go does not have traditional inheritance.
- Instead, it promotes composition.
- One struct can embed another struct to reuse fields and methods.

Example:

```
type Human struct {
    Name string
}
```

```
type Engineer struct {
    Human   // Embedding
    Lang    string
}
```

Usage:

```
e := Engineer{Human: Human{"Aman"}, Lang: "Go"}
fmt.Println(e.Name)    // Aman
fmt.Println(e.Lang)    // Go
```

6. Error Handling, Packages & Modules

→ Go's Error Handling Philosophy

- Go treats errors as values.
- Functions return an error as the last return value.
- Callers must handle errors explicitly.
- This makes code more predictable and reliable.

```
func divide(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("cannot divide by zero")
    }
    return a / b, nil
}

func main() {
    res, err := divide(10, 0)
    if err != nil {
        fmt.Println("Error:", err)
        return
    }
    fmt.Println("Result:", res)
}
```

Always check error before using result.

→ Creating Errors

1. Using errors.New()

```
import "errors"
err := errors.New("something went wrong")
```

2. Using fmt.Errorf()

```
import "fmt"
err := fmt.Errorf("invalid value: %d", val)
```

3. Error Wrapping (Go 1.13+)

```
err := fmt.Errorf("while processing user: %w", err)
```

%w wraps the error while adding more context.

4. Checking Wrapped Errors

```
if errors.Is(err, io.EOF) {
    fmt.Println("It's EOF error")
}
```

errors.Is() checks if err or any wrapped error is target.

5. Extracting Specific Error

```
var pErr *os.PathError
if errors.As(err, &pErr) {
    fmt.Println("Path:", pErr.Path)
}
```

errors.As() extracts error as target type.

→ Custom Errors

- Create your own error types by implementing error interface.
- The interface has one method: Error() string

```
type MyError struct {
    Code    int
    Message string
}

func (e MyError) Error() string {
    return fmt.Sprintf("Code: %d, Msg: %s", e.Code, e.Message)
}

func main() {
    err := MyError{Code: 500, Message: "Internal Server Error"}
    fmt.Println(err)
}
```

→ panic vs error

Feature	error	panic
Purpose	Expected failures	Unexpected failures
Handling	Should be handled	Usually not handled
Program Flow	Program continues	Program stops (unless recovered)
Example	File not found, invalid input	Index out of range, nil pointer dereference
Used For	Business logic, recoverable	Critical issues, bugs

→ Common Error Interfaces

- error is a built-in interface.
- Any type that implements Error() string satisfies it.

```
type error interface {
    Error() string
}
```

Best Practices

★ Check error as early as possible.
★ Return error up the call stack.
★ Don't ignore errors.
★ Provide meaningful error messages.
★ Use wrapping to add context.

→ Packages in Go

- A package is a collection of Go source files in the same directory.
- Every Go program consists of one or more packages.
- By default, everything in Go is unexported (private).
- To export (make public), start the name with a capital letter.

mathutil/mathutil.go

```
package mathutil

// Exported
func Add(a, b int) int {
    return a + b
}

// unexported
func multiply(a, b int) int {
    return a * b
}
```

main.go

```
package main

import (
    "fmt"
    "yourmodule/mathutil"
)

func main() {
    fmt.Println(mathutil.Add(3,4))
    // mathutil.multiply(3,4) // ✖ not accessible
}
```

Only names starting with capital letter are exported and accessible outside the package.

→ Modules in Go

- Go modules manage dependencies.
- Introduced in Go 1.11.
- File: go.mod in the root of module.

Initialize a module

```
go mod init github.com/amanviews/myapp
```

Add dependency

```
go get github.com/gin-gonic/gin
```

Tidy dependencies

```
go mod tidy
```

Removes unused dependencies and adds missing ones.

→ go.mod File

```
module github.com/amanviews/myapp

go 1.21

require (
    github.com/gin-gonic/gin v1.9.1
    github.com/google/uuid v1.6.0
)

replace example.com/old => example.com/new v1.2.0
```

module → module path
go → Go version
require → list of dependencies
replace → replace old module path with new one

→ Useful Standard Library Packages

fmt	→ Formatted I/O	path/filepath	→ File path operations	net/http	→ HTTP client & server
errors	→ Basic error handling	strings	→ String utilities	log	→ Logging
io	→ I/O primitives	time	→ Date & time	sort	→ Sorting
os	→ OS operations	encoding/json	→ JSON encoding/decoding	context	→ Cancellation & deadlines

7. Concurrency in Go: Goroutines & Channels

→ Why Concurrency in Go?

- Go is designed for concurrent programming.
- Goroutines are lightweight (very small stack ~2KB).
- Thousands of goroutines can run in the same program.
- It helps in building scalable, fast and efficient applications.

How Goroutines are Scheduled



G = Goroutine

---- Scheduler multiplexes goroutines on available OS threads.

→ Channels

- Channels are typed conduits that you can send and receive values with.
- They allow goroutines to communicate and synchronize.
- Use the chan keyword.

```
ch := make(chan int)     // unbuffered channel
ch <- 10                 // send
val := <-ch              // receive
```

Channel Operations

```
ch <- value    → send a value to channel
val := <-ch    → receive a value from channel
close(ch)      → close the channel
v, ok := <-ch  → receive with ok (checks if channel open)
```

→ Unbuffered vs Buffered Channels

Unbuffered Channel	Buffered Channel
No storage capacity.	Has fixed capacity.
Sender blocks until receiver is ready.	Sender can send until buffer is full.
Used for synchronization.	Used for decoupling and smoothing spikes.

→ Goroutines

- A goroutine is a function that runs concurrently with other functions.
- Use the keyword go to start a goroutine.
- They run in the same address space.

```
package main
import "fmt"
import "time"
func say(msg string) {
   fmt.Println(msg)
}
func main() {
   go say("Hello from goroutine!")  // starts new goroutine
   fmt.Println("Hello from main!")  // main goroutine
   time.Sleep(1 * time.Second)     // wait for goroutine
}
```

If we don't wait (Sleep / WaitGroup), main may finish before goroutine completes.

Example: Unbuffered Channel

```
ch := make(chan int)
go func() {
   fmt.Println("Goroutine: sending 42")
   ch <- 42
   fmt.Println("Goroutine: sent")
}()
val := <-ch
fmt.Println("Main: received", val)
```

Output:

```
Goroutine: sending 42
Goroutine: sent
Main: received 42
```

Example: Buffered Channel (capacity = 2)

```
ch := make(chan string, 2)
ch <- "A"
ch <- "B"          // buffer now [A B]
fmt.Println(<-ch)  // A
fmt.Println(<-ch)  // B
```

Buffer State

```
[] → [A] → [A B]
     <-ch   <-ch
     []
```

→ Range over Channel

- We can iterate over a channel using range.

```
ch := make(chan int)
go func() {
   for i := 1; i <= 3; i++ {
      ch <- i
   }
   close(ch)        // important!
}()
for v := range ch {
   fmt.Println(v)
}
```

Output:

```
1
2
3
```

→ Select Statement

- Used to wait on multiple channel operations.
- It blocks until one of the cases is ready.
- Similar to switch for channels.

```
select {
 case v := <-ch1:
    fmt.Println("Received from ch1:", v)
 case ch2 <- 10:
    fmt.Println("Sent to ch2")
 default:
    fmt.Println("No channel is ready")
}
```

→ Default Case in Select

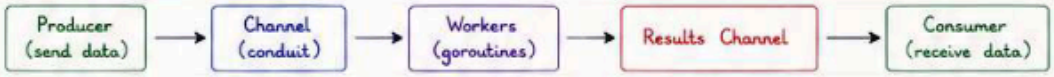
- If no case is ready, default executes immediately.

```
select {
 case v := <-ch:
    fmt.Println("Received:", v)
 default:
    fmt.Println("Nothing ready")
}
```

Important Notes

★ Don't forget to close channels when no more values will be sent.
★ Sending on a closed channel causes panic.
★ Receiving from a closed channel is safe and returns zero value.
★ Avoid goroutine leaks.

Flow: Goroutines + Channels



This pattern is the base for many concurrent designs (pipelines, worker pools, fan-in/fan-out).

8. Concurrency Patterns in Go

→ WaitGroup

- WaitGroup waits for a collection of goroutines to finish.
- Main goroutine blocks until the counter becomes zero.
- Use Add() to set the number, Done() to decrement.
- Use Wait() to block.

```
var wg sync.WaitGroup
wg.Add(2)              // number of goroutines
go func() {
    defer wg.Done()
    fmt.Println("Worker 1 done")
}()
go func() {
    defer wg.Done()
    fmt.Println("Worker 2 done")
}()

wg.Wait()              // main waits here
fmt.Println("All workers finished")
```

Add(+n) → Starts n goroutines → Each calls Done() → When 0, Wait() unblocks

→ Worker Pool

- Limit the number of concurrent workers.
- Jobs are sent to workers via a channel.
- Useful for controlling resource usage.

```
jobs := make(chan int, 100)
results := make(chan int, 100)
// workers
for w := 1; w <= 3; w++ {
  go func(id int) {
    for j := range jobs {
      fmt.Printf("Worker %d processing %d\n", id, j)
      results <- j * j     // send result
    }
  }(w)
}
// send jobs
for j := 1; j <= 5; j++ { jobs <-j }
close(jobs)
// collect results
for a := 1; a <= 5; a++ {
  fmt.Println(<-results)
}
```

Flow: Jobs → Workers (concurrent) → Results

→ Using WaitGroup with Channel

- Close channel only after all goroutines finish.
- A common and safe pattern.

```
ch := make(chan int)
var wg sync.WaitGroup
for i := 1; i <= 3; i++ {
  wg.Add(1)
  go func(n int) {
    defer wg.Done()
    ch <- n * n
  }(i)
}
// close after all workers finish
go func() {
  wg.Wait()
  close(ch)
}()
// range over closed channel
for v := range ch {
  fmt.Println(v)
}
```

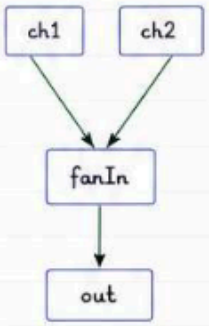
Workers send on channel → WaitGroup waits → Channel closed → Receiver ranges safely

→ Fan-in (Merging Multiple Channels)

- Combine values from multiple channels into one.
- Useful when multiple sources produce data.

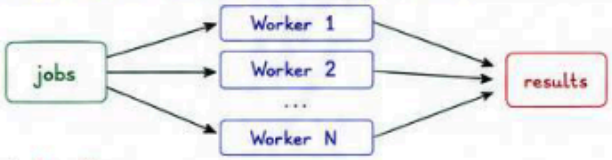
```
func fanIn(ch1, ch2 <-chan int) <-chan int {
  out := make(chan int)
  go func() {
    for v := range ch1 { out <- v }
  }()
  go func() {
    for v := range ch2 { out <- v }
  }()
  go func() {
    // wait for both to finish then close out
    runtime.Gosched()
    close(out)
  }()
  return out
}
```

Diagram



→ Fan-out (Distributing Work)

- Split work across multiple workers.
- Each worker takes from the same jobs channel.

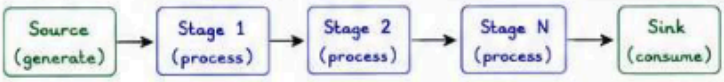


→ Benefits

- Improves throughput
- Utilizes multiple CPU cores
- Easy to scale (change number of workers)

→ Pipeline Pattern

- Breaks work into stages.
- Each stage is a goroutine.
- Output of one stage is input of next.



Close channels properly to avoid goroutine leaks.
Use context for cancellation.

→ Rate Limiting with Ticker

- Use time.Ticker to perform work at fixed intervals.
- Useful for polling, heartbeats, periodic jobs.

```
ticker := time.NewTicker(1 * time.Second)
defer ticker.Stop()
for t := range ticker.C {
   fmt.Println("Tick at", t)
}
```

Ticker.C is a channel that sends ticks.

→ Context for Cancellation

- Propagate cancellation signal across goroutines.
- Prevents goroutine leaks.

```
ctx, cancel := context.WithCancel(context.Background())
go func() {
   <-time.After(2 * time.Second)
   cancel()      // cancel after 2 sec
}()
<-ctx.Done()
fmt.Println("Cancelled:", ctx.Err())
```

ctx.Done() unblocks when cancelled.

9. Advanced Go Concepts

1. Defer, Panic, Recover

- defer runs just before the function returns.
- panic stops the normal flow and begins panicking.
- recover can stop the panic and resume normal flow.

```
func mightPanic() {
    defer func() {
      if r := recover(); r != nil {
          fmt.Println("Recovered:", r)
      }
      fmt.Println("Deferred")
   }()

   fmt.Println("Going to panic...")
   panic("something went wrong")
   fmt.Println("This will not run")
}
func main() {
   mightPanic()
   fmt.Println("Program continues...")
}
```

Output:

```
Going to panic...
Recovered: something went wrong
Deferred
Program continues...
```

2. Context Package

- Used for cancellation, timeouts, and passing deadlines.
- Widely used in APIs, DB calls, external requests.

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()

done := make(chan string, 1)

go func() {
   time.Sleep(3 * time.Second)
   done <- "task completed"
}()

select {
case res := <-done:
   fmt.Println("Result:", res)
case <-ctx.Done():
   fmt.Println("Timeout:", ctx.Err())
}
```

Use Cases:

- ★ HTTP request timeouts
- ★ Cancelling long running operations
- ★ Graceful shutdown

3. Mutex (sync.Mutex)

- Used to protect shared data from race conditions.
- Only one goroutine can access at a time.

```
var (
   mu      sync.Mutex
   counter int
)
func increment() {
   mu.Lock()       // lock
   counter++
   mu.Unlock()     // unlock
}
func main() {
   var wg sync.WaitGroup
   for i := 0; i < 5; i++ {
      wg.Add(1)
      go func() {
         defer wg.Done()
         increment()
      }()
   }
   wg.Wait()
   fmt.Println("Counter:", counter)
}
```

Note:

Use sync.RWMutex if there are more reads than writes. It allows multiple readers at the same time.

4. Atomic Operations

- For simple counters and flags without using locks.
- Package: sync/atomic

```
var counter int64

func main() {
   var wg sync.WaitGroup
   for i := 0; i < 1000; i++ {
      wg.Add(1)
      go func() {
         defer wg.Done()
         atomic.AddInt64(&counter, 1)
      }()
}
wg.Wait()
fmt.Println("Counter:", atomic.LoadInt64(&counter))
}
```

Common Functions:

- ★ AddInt64
- ★ LoadInt64
- ★ StoreInt64
- ★ CompareAndSwap

5. Race Condition

- When multiple goroutines access shared data
- concurrently and at least one writes.
- Leads to unpredictable results.

Example (Unsafe):

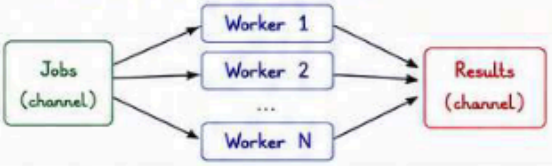
```
var x int
go func() { x = 10 }()
go func() { x = 20 }()
// Final value of x is unpredictable!
```

Run with:

```
go run -race main.go
// Detects race conditions
```

6. Worker Pool Pattern

- Fixed number of workers process jobs from a channel.
- Efficient way to limit resource usage.



Benefits:

- ★ Better control over resources
- ★ Prevents overload
- ★ Improves throughput

7. Type Conversion

- Go does not allow implicit type conversion.
- You must convert explicitly.

```
var a int = 10
var b float64 = float64(a)
var c string = strconv.Itoa(a)  // int to string
var d int, _ = strconv.Atoi("123") // string to int
```

Common Conversions:

int	→	float64
float64	→	int
int	→	string
string	→	int

8. Time Package

- Provides time handling, parsing, formatting and timers.

```
now := time.Now()
fmt.Println("Now:", now)

layout := "2006-01-02 15:04:05"
t, _ := time.Parse(layout, "2024-05-25 12:30:00")
fmt.Println("Parsed:", t)

fmt.Println("Formatted:", t.Format(layout))
time.Sleep(1 * time.Second)
```

Common Use:

- ★ Timestamps
- ★ Timeouts
- ★ Scheduling
- ★ Durations

9. Interface{} (Empty Interface)

- Can hold values of any type.
- Useful for generic programming.
- Every type satisfies empty interface.

```
func printValue(v interface{}) {
    fmt.Printf("Type: %T, Value: %v\n", v, v)
}
func main() {
    printValue(10)
    printValue("Hello")
    printValue(3.14)
}
```

Output:

```
Type: int, Value: 10
Type: string, Value: Hello
Type: float64, Value: 3.14
```

10. Best Practices & Tips in Go

1. General Best Practices

- Keep it simple and idiomatic.
- Follow standard naming conventions.
- Handle errors explicitly.
- Use gofmt and go vet.
- Write tests for your code.
- Avoid global variables.
- Prefer composition over inheritance.

Naming Conventions

- Package names → lower_case
- Function & variable → camelCase
- Exported names → PascalCase
- Constants → ALL_CAPS
- Private names → start with lowercase

2. Error Handling

- Always check errors.
- Return early on error.
- Don't ignore errors.

```
file, err := os.Open("data.txt")
if err != nil {
    return fmt.Errorf("failed to open file: %w", err)
}
defer file.Close()
```

Tip: Errors are values. Handle them like any other value.

3. Concurrency Best Practices

- Keep goroutines small and focused.
- Always close channels.
- Avoid goroutine leaks.
- Use context for cancellation.

```
ctx, cancel := context.WithTimeout(context.Background(), 2*time.Second)
defer cancel()

go func() {
    // do work
    select {
    case <-ctx.Done():
        fmt.Println("cancelled")
        return
    default:
        // continue work
    }
}()
```

Why context?
- ✓ Timeouts
- ✓ Cancellation
- ✓ Deadlines
- ✓ Request scoped values

4. Testing

- Go has built-in testing support.
- Table-driven tests are common.
- Run tests with: `go test`

```
func TestAdd(t *testing.T) {
    tests := []struct{a, b, want int}{
        {1, 2, 3},
        {2, 3, 5},
    }
    for _, tt := range tests {
        got := Add(tt.a, tt.b)
        if got != tt.want {
            t.Errorf("Add(%d,%d) = %d, want %d",
                tt.a, tt.b, got, tt.want)
        }
    }
}
```

Run:
```
go test
go test -v
go test -cover
```

5. Code Organization

- Keep packages small and focused.
- One package = one responsibility.
- Use internal/ for private code.
- Keep main() small. Move logic to functions.

```
myapp/
├── cmd/
│   └── main.go
├── internal/
│   ├── config/
│   │   └── config.go
│   └── service/
│       └── service.go
├── pkg/
│   └── utils/
│       └── helper.go
├── go.mod
└── README.md
```

- cmd/ → executable
- internal/ → private (not importable outside module)
- pkg/ → reusable public packages

6. Performance Tips

- Avoid unnecessary allocations.
- Reuse slices with capacity.
- Use sync.Pool for temporary objects.
- Profile your code.

```
buf := make([]byte, 0, 1024)  // pre-allocate
for _, item := range items {
    buf = append(buf, process(item)...)
}
```

Profile:
```
go tool pprof
 cpu.prof

go test -cpuprofile
 cpu.prof
```

7. Common Pitfalls

- Don't write to closed channel.
- Don't range over nil channel (blocks forever).
- Don't forget to close resources.
- Avoid empty interface{} unless necessary.

Avoid	Prefer
var ch chan int	ch := make(chan int)
interface{}	specific types
time.Sleep() for sync	channels / sync.primitives
Global variables	dependency injection

8. Useful Commands

Command	Purpose
go run main.go	Run a Go program
go build	Build the binary
go test	Run tests
go test -cover	Run tests with coverage
go vet ./...	Examine source code
go fmt ./...	Format code
go mod tidy	Clean up dependencies

9. Golden Rules

- ★ Do one thing and do it well.
- ★ Make it work. Make it right. Make it fast.
- ★ Clear is better than clever.
- ★ Concurrency is powerful, use it wisely.
- ★ Read the docs. They are excellent!

Keep learning,
Keep building,
Happy Coding! ☺

Go is simple, fast, reliable and efficient.
Write clean code. Handle errors. Use goroutines wisely. Build scalable systems. Enjoy the journey!